

# PROJECT 5: Clinic SOP Copilot with MCP

## Tool-based Agent via Model Context Protocol

### Scenario

Clinics need SOP-consistent answers (e.g., “How do we prep a patient for MRI with contrast?”).

Build an MCP server exposing:

- a *filesystem tool* (reads sops/\*.md)
- a *policy QA tool* (structured search over SOP headings)
- a *web tool* (Serper search for reputable references)

**Primary stack:** MCP (server + client), your LLM of choice, Opik for eval

### MCP Tools

- `sop.search(query)` → returns section IDs & excerpts
- `sop.read(section_id)` → returns full section
- `web.search(query)` → Serper JSON results (title/url/snippet)
- `cite.validate(text, citations)` → simple checker to ensure each claim has a source

### Client

- Minimal CLI or VS Code or Claude Desktop integrated sample that calls MCP tools before responding

### Deliverables

- `mcp-server/` (Node/TS or Python)
- `client/` with a “force-tool-use” system prompt and conversation log
- `tests/` with probe questions → expected section IDs & allowed answers

### Evaluation (Opik)

- *Tool Compliance*: % turns that used a tool before answering
- *Citation Coverage*: % sentences with valid citation anchors
- *Answer Consistency*: exact-match scores vs. canonical SOP snippets

### Team of 4 split

1 MCP server/tools, 1 client & UX, 1 SOP content & indexing, 1 eval/Opik hooks

### Stretch

- Add guardrails: block answers if no SOP match  $\geq$  threshold
- Add “uncertainty mode” that asks the user to clarify

## **SOPs dataset**

Synthetic SOPs are provided

### **How to use with MCP tools**

- **sop.search(query)**: search titles, keywords, and [SECTION: ...] blocks. Return section IDs like SOP-001::PROCEDURE.
- **sop.read(section\_id)**: map SOP-XXX::SECTION\_NAME → full section text.
- **cite.validate(text, citations)**: ensure answers include anchors such as SOP-001 [SECTION: PROCEDURE].